TAU - Advanced Topics in Programming, Semester B 2026

Assignment 1 - v2

Note that the draft period ended: commenting on the document is closed. All questions on the assignment should be submitted in the relevant forum in the course site! Changes in the doc, if any, would be highlighted in the document and a proper message will be shared via the course site. Note: if a change creates more work for you (e.g. you already implemented things differently), please raise the issue in the assignment forum before rushing to change your code!

Submission deadline: May ~~17th~~ 20th, 23:30.
Note: here are the Submission Instructions - please make sure to follow them!

Assignment 1 - Requirements and Guidelines

You are required to implement a program that simulates a drone mapper.

In this first assignment the requirements are a bit open, you will have to select your API, class design and input file format. Note that in the next assignments a common API and file formats will be dictated and you will have to refactor your code accordingly.

Drone Mapper

The Drone Mapper is a drone designed for mapping 3D buildings, by flying inside and scanning the surroundings. In this exercise you are required to implement the SW parts of it.

The Drone Mapper operates as follows:

- The drone starts at a certain location {X, Y, Height}, indicating the center of the drone and an initial XY-Angle which is zero by default (0 = “east” on the X axis, 90 = “south” on Y axis, 180 = “west” on the X axis, 270 = “north” on the Y axis).
- The drone moves through the 3D space to map its surroundings.
- The drone has an internal ‘Position Sensor’ that provides its exact center position, in terms of {X, Y, Height and XY-Angle}.
- The drone has an internal ‘Lidar Sensor’ that provides information about elements (walls, ceiling, floor, obstacles) in the direction it points to, within the field of view (FOV) given in the configuration below.
- The drone movement driver has the commands: ‘Rotate <Left/Right> by <angle>’, ‘Advance <distance amount>’ and ‘Elevate <distance amount>’. Note that angle and distance amount can be positive or negative.
- The drone should manage the mapping mission autonomously, stopping when the entire mapping is complete and it’s back in its initial position where it started.
- The output of the mapping is a sparse 3D matrix, with an API for allowing inspection for any {X, Y, Height}, resulting with: 0 if this position is empty, 1 if it is occupied, -1 if it is not mapped because it couldn’t be mapped and -2 if it is not mapped because it’s not within the required mapping boundaries.
- There is an initial configuration, specifying the drone’s capabilities, which includes:
  - The minimal dimension it can pass through, to avoid attempts to enter overly narrow entries (in terms of width, length and height). In Exercise 1, you may assume that the drone is a perfect sphere.
  - Max movement per request, for: Rotate (angle), Advance and Elevate (distance).

- Lidar capabilities: The lidar always emits one beam directly along the center of its field of view (FOV). This central beam is referred to as “Circle 0”. In addition, the lidar emits additional beams from the same origin, pointing towards outer circles. The lidar configuration defines:
  - Z-min: The minimum operational distance from the lidar (e.g., 20 cm), below this distance the beams can identify objects, but cannot accurately measure their distance from the lidar.
  - Z-max: The maximum operational distance from the lidar (e.g., 120 cm). Beyond this range, the lidar cannot identify objects.
  - D: The spacing between consecutive circles of beams at distance Z-min (e.g. 2.5 cm = “Circle 1” radius is 2.5 cm, “Circle 2” radius is 5 cm, etc., when measured 20 cm from the lidar). Each outer circle has 4 times the amount of beams than the previous one. The beams in each circle are evenly spread around the circle’s circumference and aligned with the lidar’s elevation angle, ensuring symmetric coverage.
  - FOVC: number of beam circles (e.g.: 1 = there is only one single central beam. 5 = there are 5 circles, indexed 0 to 4, with a total of 341 beams).

  It’s important to note that the lidar beams may have blind spots, areas that fall in the gaps between adjacent beams will not be detected by the lidar. It’s your responsibility to make sure that you do not ignore such areas unless their size is below the required resolution, as set by the mission configuration.

- There is an additional configuration, specific for the mission, which includes:
  - The mapping boundaries, in terms of:
    - bounded rectangle, in terms of: min X, max X, min Y, max Y.
    - max and min height.
  - The initial position of the drone, in terms of {X, Y, Height} indicating the center of the drone, and an initial XY-Angle. If angle is not given assume 0 (“east”).
  - The required resolution of the result mapping, in terms of the number of decimal places after the dot, for X and Y and separately for Height.

    NOTE: In Ex1 you may assume that there is a certain single supported resolution. Any requested mapping resolution that is not the one supported may yield an error. The supported resolution can be the resolution of the map provided to the simulation (and from there to the MockLidar).
  - Recharging positions, given as a set of {X, Y, Height}, the set may be empty.
- All units shall be based on Strong Types, as explained below.

Your project shall create mock sensors and engine driver, to allow simulating the drone’s algorithms in a SW only environment.

Lidar Scan Results

The results of each lidar scan is a data structure that includes, for each beam that hits an object within the scan range: The 3D-angle of the hitting beam (azimuth) and the distance to the detected object. If the detection distance is below Z-min, the distance value returned for this beam is 0, indicating a hit that is too close to be measured accurately.

The data structure size cannot exceed the number of beams.

The data structure size can be zero, if no beam detected any object in its direction, up to Z-max distance. Note that there still may be objects in blind spots between beams within this distance. NOTE: In Ex1 you may assume and implement a lidar that returns info on all beams.

Input

The program should get as an input:

1. A text file for the drone's capabilities.
2. A text file for the specific mission. In exercise 1 this will NOT include recharging positions and you should assume that the drone has infinite battery.
3. A text or binary file for simulating the map of the building to be mapped by the drone. This map is not visible to the drone and serves ONLY to allow the Lidar mock sensor to provide information about the building.

It is your responsibility in exercise 1 to decide about the formats of the files.

Output

The program should generate an output file in the same format as the file provided for mapping the building. Then it should compare the two files and give a score for the mapping, between 0 and 100. Note that there can be cases where the score cannot reach 100, if there are parts of the building which the drone cannot access.

It is your responsibility in exercise 1 to decide about the score formula.

Flow

The program is in fact a simulator for the drone algorithm.
When it starts it shall load the input files provided, then initialize the drone with:

- Lidar Sensor interface (providing actually a Lidar mock, drone is not aware of that)
- Drone Position Sensor interface (again, a mock, again drone is not aware of that)
- Drone Movement Driver interface (yet again a mock, drone is not aware of that either)
- Building Map interface with the API for setting and getting each {X, Y, Height}, with the values {0: empty, 1: occupied, -1: not mapped, -2: beyond boundaries}

(Note: it might be that the 3 mocks provided above need to interact between them somehow).

Then there shall be a "main loop" - while mapping is not complete:

- Ask the drone for a "command" from:
    - Rotate (Left / Right) <angle>
    - Advance <distance amount>
    - Elevate <distance amount>
    - Scan <X-Y angle in reference to current direction, height angle>
        - The default angle for X-Y if not provided is the drone direction
        - The default height angle if not provided is zero
    - Get Location
    - Finished - Note: in Ex1 you can report "Finished" when you completed the mapping, no need to return to the starting position!
- Drone can get and set values into the provided Building Map interface. Note that the values retrieved from the map are based ONLY on values provided previously by the drone and are NEVER based on the mapping provided to the simulation as an input.

When the loop finishes, the program shall write the Building Map info into file, compare it with the simulation building info file and print to screen the score of this run.

Algorithm

You should implement the drone algorithm. In Ex1 it is OK if the algorithm is not optimized, but it should still aim for completing the mapping successfully based on algorithms such as BFS or DFS etc. All decisions must be deterministic (without any randomness).
The algorithm shall never cause the drone to collide into elements (this would end the simulation with a proper failure notice).

Error Handling

You should reasonably handle any error:

- The program shall not crash, ever.
- You should better recover from input file errors if possible, by getting a reasonable default decision to replace missing or bad values. You should write a short description of all recovered errors found in the input files, into input_errors.txt file. Create this file only if there are errors.
- In case there is an unrecoverable error, a message should be printed to screen before the program finishes. No need for input_errors.txt file in this case.
- You should not end the program using the “exit” function or similar functions, even in error cases. The program shall end by finishing main, in all scenarios.

Running the Program

drone_mapper [<input_output_files_path>]

If the argument <input_output_files_path> is missing then the path to use would be the current working directory.

The program will search in the <input_output_files_path> for three files:

- drone_config.txt
- mission_config.txt
- map_input.txt

The program will create an output file (overwrite if already exists):

- map_output.txt

It is your responsibility in exercise 1 to decide about the formats of the files.

Read also the additional submission instructions which are part of the requirements!

Strong Types

In input and output files: angles are in degrees (0-360), distance/length values are in cm.
In code: all values and APIs shall use the mp-unit library, with the use of:

```
using si::unit_symbols::m;   // meters
using si::unit_symbols::cm;  // centimeters
using si::unit_symbols::deg; // degrees
```

Code example: https://godbolt.org/z/hr6srz88M

Additional required documents (MANDATORY - part of the assignment grade!):
Add a short high level design document (HLD) as a PDF file, containing:

- A UML **class diagram** of your design.
- A UML **sequence diagram** of the main flow of your program (only main flow).
- Explanations of your design considerations and alternatives.
- Explanations of your testing approach.

Bonuses
You may be entitled for bonus points for interesting implementation.
NOTE that implementing a smart algorithm will not be entitled for a bonus, as this would be part of your next assignments.
But, the following may be entitled for a bonus:

- adding logging, configuration file or other additions that are not in the requirements.
- having automatic testing, based on GTest for example.
- adding visual simulation (do not make it mandatory to run the program with the visual simulation, you may base the visual simulation on the input and output files as an external utility, it can be written in C++ or in another language).

BONUSES IMPORTANT NOTE: In order to get a bonus for any addition, you MUST add to your submission, inside the main directory, a *bonus.txt* file that will include a listed description of the additions for which you request for a bonus and how to check them.

cont' next page

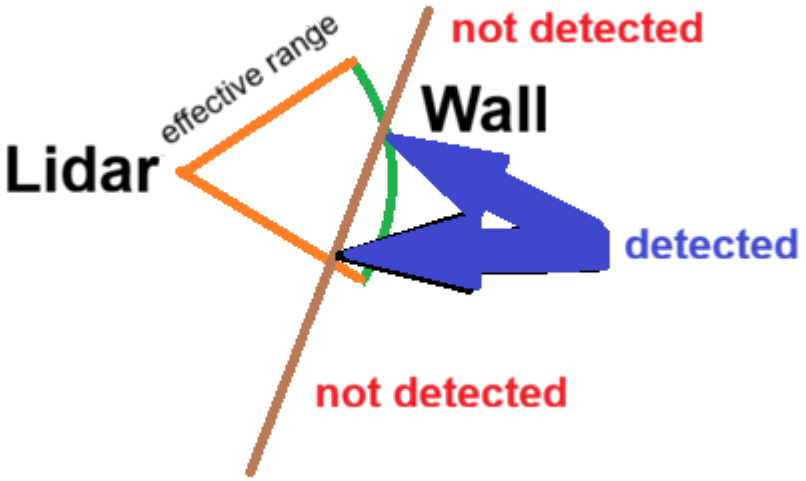
FAQ

Q:
Can we use AI when working on the assignments?

A:
You may (and even should) use GenAI tools, but remember that the responsibility for meeting the requirements and submitting working code is on you. Moreover, you are required to be able to explain any part of the code that you submit, this will be checked in short zoom meetings set with students separately, after your submission. You must read carefully and understand any piece of code that you integrate into your submission and be able to fully explain it.

Q:
How does the FOV and effective range affect the lidar scan results?

A:
The Lidar cannot detect objects beyond its effective range or outside the FOV angle.
Below is a 2D diagram of the lidar scan view.
In the diagram there are two sections of the brown wall that are not detected: the top section is not detected since it's beyond the effective range of the lidar, the bottom section is not detected since it's outside of the lidar FOV.
To imagine the actual 3D view, assume we rotate the orange triangle to create a cone, everything that is within the cone's limits and the effective range would be detected.
Note that the reported distance per each 'cell' of the wall in front of the lidar will be different, according to the actual distance from the lidar.



cont' next page

Q:
Is it guaranteed that the lidar will detect all objects within its effective range?

A:
No. The lidar has “blind spots” between beams. For short distances these blind spots might be very small (and thus negligent, in terms of the required mapping resolution) but it might be that at a certain range which is still within the effective range of the lidar, there would be blind spots that will require getting closer or changing the lidar azimuth for further scanning.

Q:
Let’s assume that we mapped two “cells” (based on the required mapping resolution) as occupied, and there is a single cell between them which is in a blind spot. Can we interpolate and assume it’s also occupied?

A:
No. You should actively map all “cells” in the required mapping resolution.

Q:
Let’s assume that we mapped two “cells” (based on the required mapping resolution) as empty, and there is a single cell between them which is in a blind spot. Can we interpolate and assume it’s also empty?

A:
No. You should actively map all “cells” in the required mapping resolution.

Q:
Assume there is an object in front of a lidar beam, and another object in the background, hidden behind the object that is in front. Would the object that is hidden behind be detected by the lidar?

A:
Each lidar beam can only identify one object, thus only the front object will be detected. If the front object is smaller than the one behind, it might be that another beam will detect parts of the object that is in the background.